



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 3

FPGA — INTRODUCTION TO ALTERA QUARTUS II
SOFTWARE, AND INTERFACING FPGA TO DIP SWITCH AND
SEVEN SEGMENT

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 3

Introduction to Altera Quartus II Software, and Interfacing FPGA to DIP Switch and Seven Segment

Objectives

1. To use schematic entry as a design method in Altera Quartus II.
2. To use Verilog as a design entry method in Altera Quartus II.
3. To combine schematic and Verilog as a single design entry in Altera Quartus II.
4. To simulate the circuit using Altera Quartus II.
5. To download the design onto an Altera DE2 board.
6. To interface a DIP switch and a seven-segment display to the FPGA.

Equipment / Software Required

1. Altera Quartus II software
 - version 9/10/12 for Cyclone II
 - version 9/10/12/latest for Cyclone IV
2. Altera DE2 board
3. Function generator

Introduction

Seven-segment displays are widely used to show numbers. The DE2 board provides eight seven-segment displays, 18 DIP switches, four push-button switches, and 16 LEDs. These components are hardwired on the board and are each assigned to specific FPGA pins — refer to the DE2 user manual for the full pin map.

Procedure

Activity 1 — Schematic entry

Objective: use schematic entry to design a 4-to-1 multiplexer in Quartus II.

1. Open Quartus II and start the New Project Wizard.
 - Working directory: fpga; project name: activity1; top-level entity: mux_sch.
 - Device family: Cyclone II (EP2C35F672C6N) or Cyclone IV (EP4CE115F29C7N).

- Accept the default EDA tool settings (synthesis, simulation, timing analysis) and finish.
2. Create a new Block Diagram/Schematic file.
 3. Place three 3-input AND gates and one 4-input OR gate from the primitives library, and wire them as shown in Figure 3.1 using rubberbanding to connect components.
 4. Add and name the input pins (i0–i3, s0, s1) and the output pin (out).
 5. Save as mux_sch.bdf and start compilation.
 6. Create a Vector Waveform File to simulate the design: insert input nodes i0–i3, s0, s1, and output node out.
 7. Assign logic values to the inputs, save as mux_sch.vwf, then run Compilation and Simulation.
 8. Record the outputs, then repeat for each input combination in Table 3.1, saving the waveform file each time.
 9. Print and submit the schematic diagram and simulation result.

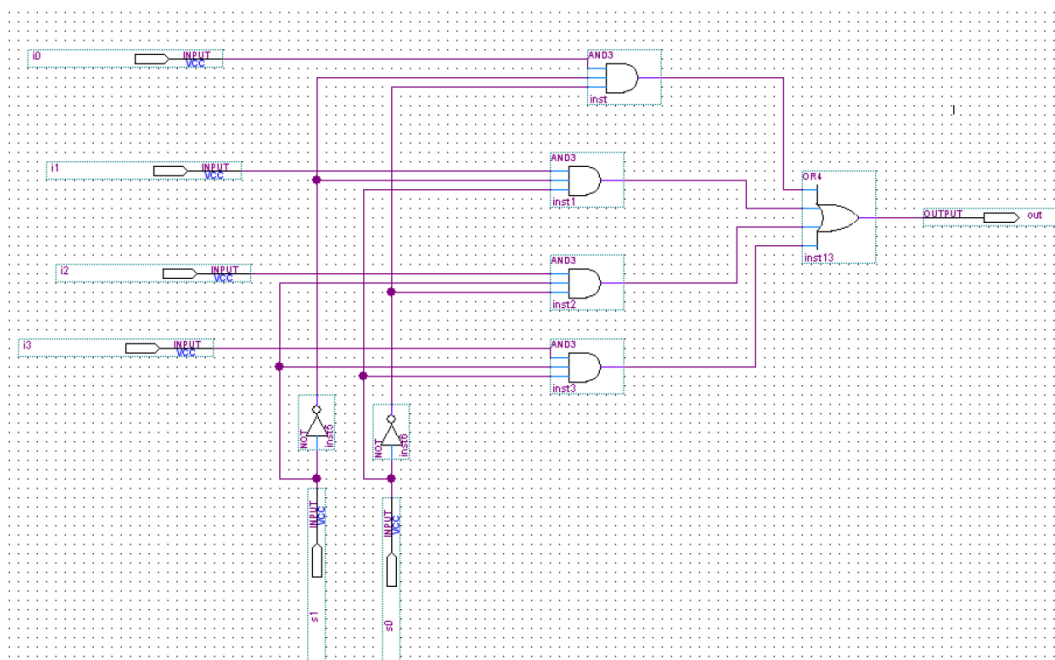


Figure 3.1: Mux Schematic.

Table 3.1: Input combinations to simulate.

S0	S1	I0	I1	I2	I3
0	0	1	1	0	0
0	1	0	1	1	0
1	0	1	1	0	1
1	1	1	1	0	0

Activity 2 – Verilog entry

Objective: use Verilog code as a design entry method in Quartus II.

1. Start a new project (Activity2, top level mux_ver) with the same device and EDA tool settings as Activity 1.
2. Create a new Verilog file and enter the code in Listing [3.1](#).
3. Save as mux_ver, then compile the design.



4. Create a Vector Waveform File; insert input nodes i0–i3, s0, s1, and output node out.
5. Assign logic values matching Table 3.1, save as mux_ver.vwf, then start simulation.
6. Inspect the result in the Simulation Report — Simulation Waveform.
7. Change the input values (saving before each run) and re-simulate.
8. View the synthesized schematic via **Tools** → **Netlist Viewer** → **RTL Viewer**, shown in Figure 3.2.
9. Print and submit the Verilog code and simulation result.

```

mux_ver.v
1  module mux_ver(out, i0, i1, i2, i3, s1, s0);
2      output out;
3      input i0, i1, i2, i3;
4      input s1, s0;
5      reg out;
6
7      always @(s1 or s0 or i0 or i1 or i2 or i3)
8      begin
9          case ({s1, s0})
10             2'b00: out = i0;
11             2'b01: out = i1;
12             2'b10: out = i2;
13             2'b11: out = i3;
14             default: out = 1'bx;
15         endcase
16     end
17 endmodule

```

Listing 3.1: mux_ver.v — Verilog 4-to-1 multiplexer.

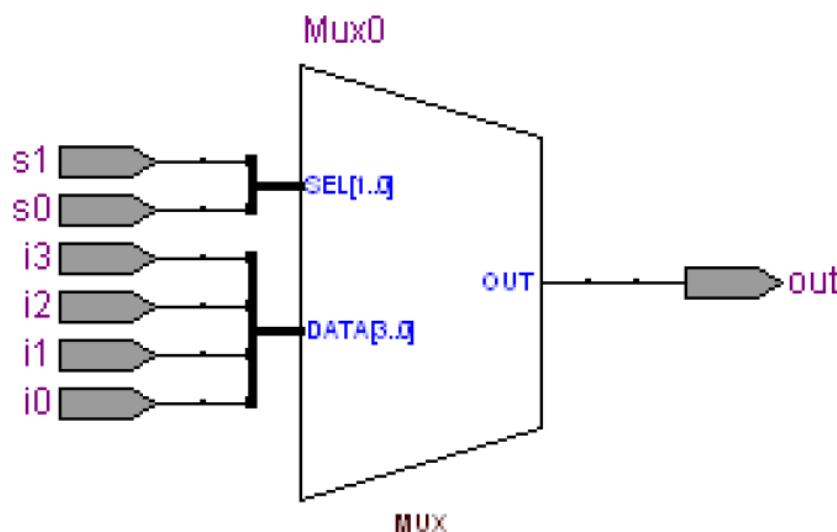


Figure 3.2: RTL Viewer output for mux_ver (Mux0).

Activity 3 — Combining schematic and Verilog entry

Objective: combine a Verilog module with schematic entry in a single design.

1. Start a new project (activity3, top level mux_schver) with the same device and EDA tool settings.
2. Create a new Verilog HDL file with the code in Listing 3.2; save as mux_vr1.
3. Generate a reusable symbol from the file via **File** → **Create/Update** → **Create Symbol Files for Current File**. The resulting block is shown in Figure 3.3.
4. Create a new Block Diagram/Schematic file, place the mux_vr1 symbol from the project library, and connect its inputs and output to input/output pads as shown in Figure 3.4.
5. Save as mux_schver.bdf and start compilation.
6. Create a Vector Waveform File, insert the same nodes as before, assign logic values, save as mux_schver.vwf, and run Compilation and Simulation.
7. Record the outputs for the combinations in Table 3.1, saving the waveform file each time.
8. Print and submit the schematic diagram, Verilog code, and simulation result.

```
mux_vr1.v
1  module mux_vr1(out, i0, i1, i2, i3, s1, s0);
2      output out;
3      input i0, i1, i2, i3;
4      input s1, s0;
5      reg out;
6
7      always @(s1 or s0 or i0 or i1 or i2 or i3)
8          begin
9              case ({s1, s0})
10                 2'b00: out = i0;
11                 2'b01: out = i1;
12                 2'b10: out = i2;
13                 2'b11: out = i3;
14                 default: out = 1'bx;
15             endcase
16         end
17     endmodule
```

Listing 3.2: mux_vr1.v — Verilog module to be reused as a schematic symbol.

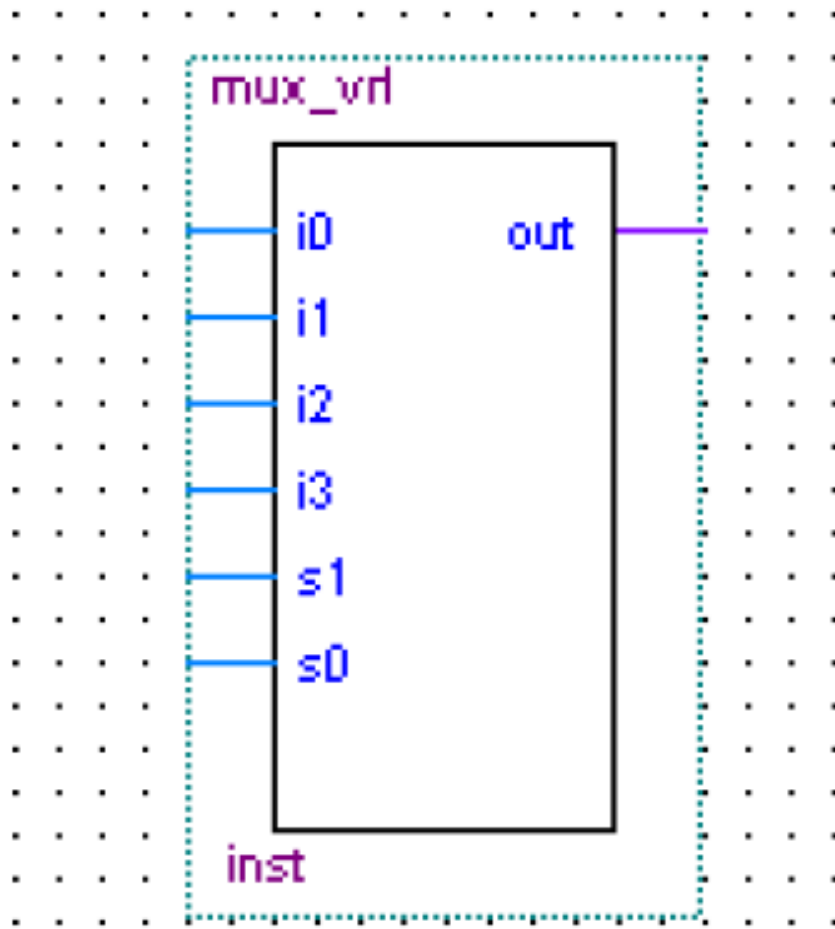


Figure 3.3: Auto-generated mux_vr1 symbol block.

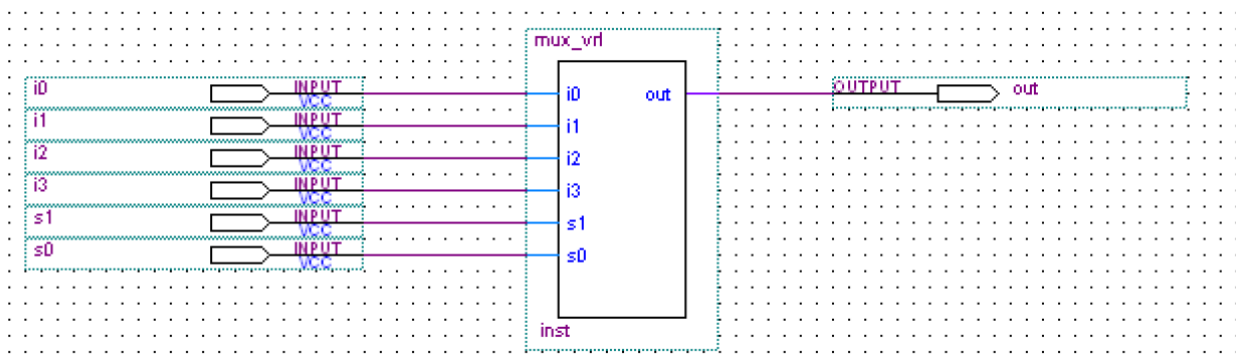


Figure 3.4: Combined schematic mux_schver.

Activity 4 – Timing analysis

Objective: analyse the maximum operating speed of a design.

1. Start a new project (Activity4, top level counter) with the same device and EDA tool settings.
2. Create a new Verilog file with the code in Listing 3.3; save as counter and compile.
3. Create a Vector Waveform File; insert cLock and cClear as inputs and Q as output.

4. Force `clear` low, assign a clock signal to `clock`, save as `counter.vwf`, then start simulation.
5. Inspect the result in the Simulation Report — Simulation Waveform.
6. Run **Processing** → **Start** → **Start Classic Timing Analyzer**.
7. Read the maximum clock frequency from the Timing Analyzer Summary (Figure 3.5).
8. Print and submit the Verilog code and simulation result.

```

counter.v
1  module counter(Q, clock, clear);
2      output [3:0] Q;
3      input  clock, clear;
4      reg [3:0] Q;
5
6      always @(posedge clear or negedge clock)
7      begin
8          if (clear)
9              Q = 4'd0;
10         else
11             Q = (Q + 1) % 16;
12     end
13 endmodule

```

Listing 3.3: `counter.v` — 4-bit up counter with asynchronous clear.

Timing Analyzer Summary									
	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths
1	Worst-case tco	N/A	None	7.410 ns	Q[3]~reg0	Q[3]	clock	--	0
2	Clock Setup: 'clock'	N/A	None	Restricted to 233.64 MHz (period = 4.280 ns)					
					Q[0]~reg0	Q[3]~reg0	clock	clock	0
3	Total number of failed paths								0

Figure 3.5: Timing analyzer summary for the counter design.

Activity 5 — Downloading to the DE2 board

Objective: download a Verilog design onto the Altera DE2 board and drive the seven-segment display.

1. Start a new project (activity5, top level counta) with the same device and EDA tool settings.
2. Create a new Verilog HDL file with the code in Listing 3.4; save as `counta`.
3. Compile and simulate the design.
4. Assign each output pin to a physical device pin using **Assignment** → **Pin Planner**, per Table 3.2 (refer to the DE2 user manual for pin locations).
5. Save, then recompile the project.
6. Download the configuration to the DE2 board via **Tools** → **Programmer** → **Hardware Setup** → USB-Blaster → **Start**.

7. Record the pattern displayed on the seven-segment display.
8. Change the assignment to assign `q[6:0] = 7'b1111000;`, save, recompile, and download again. Record the new displayed pattern.

```
counta.v
1 module counta(q);
2   output [6:0] q;
3   assign q[6:0] = 7'b0010000;
4 endmodule
```

Listing 3.4: `counta.v` — static seven-segment pattern.

Table 3.2: Seven-segment output pin assignment (DE2 board).

#	Node name	Direction	Pin
1	Q[6]	Output	
2	Q[5]	Output	
3	Q[4]	Output	
4	Q[3]	Output	
5	Q[2]	Output	
6	Q[1]	Output	
7	Q[0]	Output	

Activity 6 — 16-bit binary counter with frequency divider

Objective: design a 16-bit binary counter driven through a frequency divider.

1. Open Quartus II and write the frequency-divider module in Listing 3.5 (seven), which lowers the incoming clock frequency (Device family: Cyclone II (EP2C35F672C6N) or Cyclone IV (EP4CE115F29C7N)).
2. Create a reusable symbol for seven.
3. Write the 4-bit counter module in Listing 3.6 (led).
4. Create a reusable symbol for led.
5. Draw the schematic in Figure 3.6, connecting the seven block's most significant output bit `q[16]` to the clock input of the led block.
6. Assign pin numbers as shown in Table 3.3 (refer to the DE2 user manual).
7. Use a function generator to supply the clock signal, using 1 MHz as the clock input.
8. Determine the effective input frequency reaching the led module.
9. Discuss your result based on the observed LED output display.

seven.v — frequency divider

```

1 module seven(q, clk);
2   output [15:0] q;
3   input clk;
4   reg [15:0] q;
5
6   always @(posedge clk)
7   begin
8     q = (q + 1) % 65536;
9   end
10 endmodule

```

Listing 3.5: seven.v — 16-bit frequency divider.

led.v — 4-bit counter

```

1 module led(x, clk);
2   input clk;
3   output [3:0] x;
4   reg [3:0] x;
5
6   always @(posedge clk)
7   begin
8     x = (x + 1) % 16;
9   end
10 endmodule

```

Listing 3.6: led.v — 4-bit binary counter.

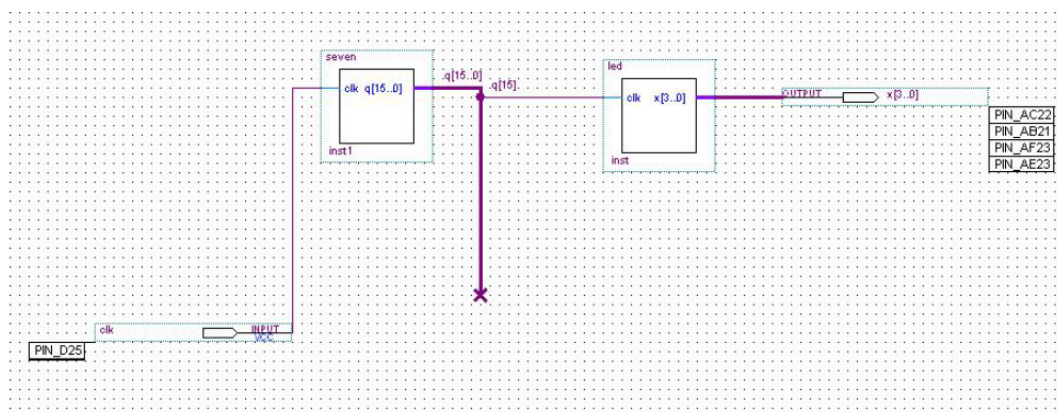


Figure 3.6: Schematic diagram for the 16-bit binary counter with frequency divider.

Table 3.3: Pin assignment for the 4-bit binary counter.

#	Node name	Direction	Pin location
1	clk	Input	
2	X[3]	Output	
3	X[2]	Output	
4	X[1]	Output	
5	X[0]	Output	

Activity 7 – 0–99 counter on seven-segment display

Objectives: (1) design a counter that counts 0 to 99; (2) display the result on the seven-segment displays.

1. Using a DIP switch as the trigger input, write Verilog code for a counter that counts from 0 to 99 once the switch is triggered.
2. Decode and display the result (tens and units digits) on two seven-segment displays.
3. Print and submit the Verilog code and simulation result.

Activity 8 – Running light

Objective: design a running-light display on the Altera DE2 board.

1. Using a DIP switch as the input, write Verilog code to implement a running-light (chasing LED) pattern.
2. Use three DIP switches to select between three different running-light patterns.
3. Print and submit the Verilog code and simulation result.